Design and Analysis of Computer Algorithms

01:198:344

Spring 2026, Rutgers

Instructor: Surya Teja Gavva

Last Class

- Merge sort: Sort left, sort right, merge in $O(n)$.
  Recurrence: $T(n) = 2T(n/2) + O(n)$
  Expanding $n \to n/2 \to n/4$ gives $T(n) = O(n \log n)$
- Karatsuba's method: Divide and conquer reduces multiplication of n-digit numbers to $n/2$.
  Standard: $T(n) = 4T(n/2) + O(n)$
  An algebraic identity reduces one multiplication:
  $T(n) = 3T(n/2) + O(n)$, solving gives $T(n) = O(n^{\log_2 3})$
- Strassen's matrix multiplication.
  Standard: $T(n) = 8T(n/2) + O(n^2)$
  Strassen's trick reduces multiplications to 7:
  $T(n) = 7T(n/2) + O(n^2)$, solving gives $T(n) = O(n^{\log_2 7})$
- Recursion tree method: Expand recurrence, track work at each level.
  Three cases from summing a geometric series:

$$S = a + ar + ar^2 + \ldots + ar^{k-1} = \begin{cases} O(a) & r < 1 \\ O(ka) & r = 1 \\ O(ar^k) & r > 1 \end{cases}$$

Recursion Tree Expansion: Case 1

Solving the recurrence:

$$T(n) = 3T(n/4) + n$$

Expanding step by step:

$$\begin{aligned}
T(n) &= 3T(n/4) + n \\
T(n) &= 3\left[3T(n/4^2) + n/4\right] + n \\
&= 3^2 T(n/4^2) + \frac{3}{4}n + n \\
T(n) &= 3^3 T(n/4^3) + \frac{3}{4^2}n + \frac{3}{4}n + n \\
T(n) &= 3^k T(n/4^k) + n\left(1 + \frac{3}{4} + \frac{3}{4^2} + \cdots + \frac{3}{4^{k-1}}\right)
\end{aligned}$$

Let $n = 4^k \Rightarrow k = \log_4 n$, then: $T(n) = 3^{\log_4 n} T(1) + n\left(1 + \frac{3}{4} + \frac{3}{4^2} + \cdots + \frac{3}{4^{\log_4 n - 1}}\right)$

Since the geometric series has ratio $\frac{3}{4} < 1$, the sum converges to a constant, we have $T(n) = \Theta(n)$

Recursion Tree Expansion: Case 2

Solving the recurrence:

$$T(n) = 4T(n/4) + n$$

Expanding step by step:

$$\begin{aligned}
T(n) &= 4T(n/4) + n \\
T(n) &= 4\left[4T(n/4^2) + n/4\right] + n \\
&= 4^2 T(n/4^2) + \frac{4}{4}n + n \\
T(n) &= 4^3 T(n/4^3) + \frac{4}{4^2}n + \frac{4}{4}n + n \\
T(n) &= 4^k T(n/4^k) + n\left(1 + \frac{4}{4} + \frac{4}{4^2} + \cdots + \frac{4}{4^{k-1}}\right)
\end{aligned}$$

Let $n = 4^k \Rightarrow k = \log_4 n$, then:

$$T(n) = 4^{\log_4 n} T(1) + n \sum_{i=0}^{\log_4 n - 1} 1$$

Since the sum grows as $O(\log n)$, we get:

$$T(n) = O(n \log n)$$

Recursion Tree Expansion: Case 3

Solving the recurrence:

$$T(n) = 5T(n/4) + n$$

Expanding step by step:

$$\begin{aligned}
T(n) &= 5T(n/4) + n \\
T(n) &= 5\left[5T(n/4^2) + n/4\right] + n \\
&= 5^2 T(n/4^2) + \frac{5}{4}n + n \\
T(n) &= 5^3 T(n/4^3) + \frac{5}{4^2}n + \frac{5}{4}n + n \\
T(n) &= 5^k T(n/4^k) + n\left(1 + \frac{5}{4} + \frac{5}{4^2} + \cdots + \frac{5}{4^{k-1}}\right)
\end{aligned}$$

Let $n = 4^k \Rightarrow k = \log_4 n$, then:

$$T(n) = 5^{\log_4 n} T(1) + n \sum_{i=0}^{\log_4 n - 1} \left(\frac{5}{4}\right)^i$$

Since the sum is $O((5/4)^k)$, we get:

$$T(n) = O(n^{\log_4 5})$$

Introduction to Recursion Tree Method

- The recursion tree method is a way to visualize the work done by a recursive algorithm.
- Each node in the tree represents the cost of a subproblem.
- The total work is the sum of the work at all levels of the tree.
- Useful for solving recurrence relations of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

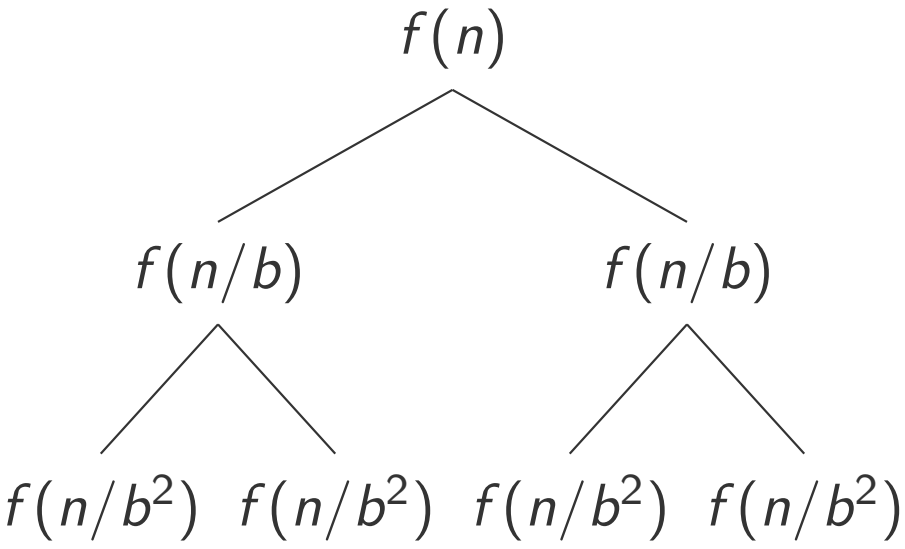
where $a \geq 1$, $b > 1$, and $f(n)$ is the work done at each level.

Visualizing Work at Recursion Levels

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- The recursion tree has:
  - **Root**: Work $f(n)$.
  - **Level 1**: a nodes, each with work $f(n/b)$.
  - **Level 2**: a^2 nodes, each with work $f(n/b^2)$.
  - **...**
  - **Level** k: a^k nodes, each with work $f(n/b^k)$.
- The tree has $\log_b n$ levels.

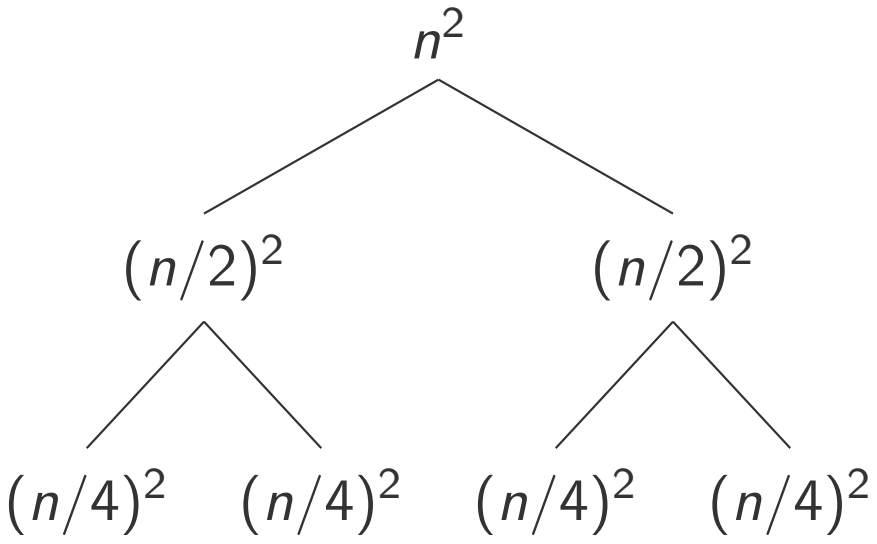
Recursion Tree Visualization



- Each level represents a step in the recursion.
- The work at each level is the sum of the work in all nodes at that level.

Example: Decreasing Work

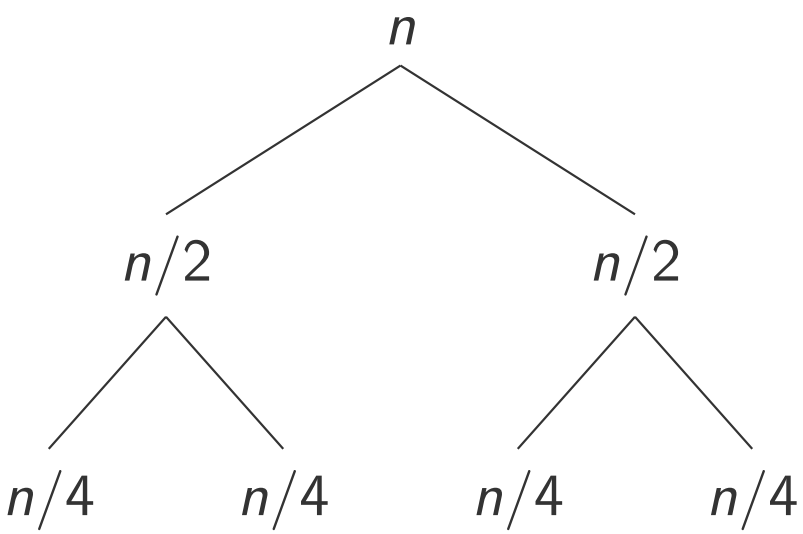
- Consider $T(n) = 2T(n/2) + n^2$.
- Work at each level decreases geometrically: $n^2, n^2/2, n^2/4, \cdots$



- Total work dominated by the root level: $O(n^2)$.

Example: Equal Work

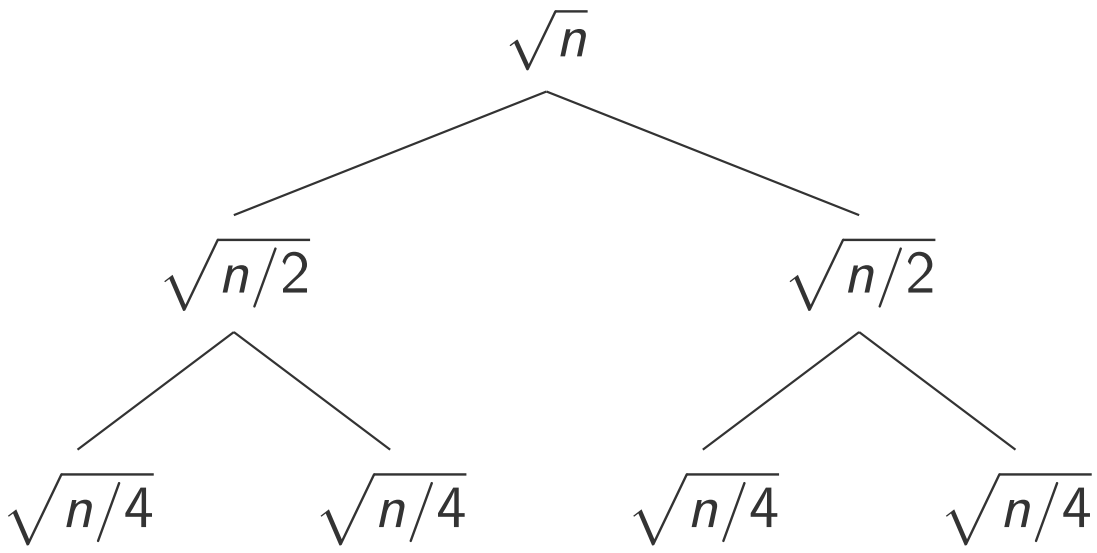
- Consider $T(n) = 2T(n/2) + n$.
- Work at each level is equal: $n, n, n, \cdots$



- $O(\log n)$ levels and so the total work is $O(n \log n)$.

Example: Increasing Work

- Consider $T(n) = 2T(n/2) + \sqrt{n}$.
- Work at each level increases geometrically:



- Work at level 0: $\sqrt{n}$.
- Work at level 1: $\sqrt{2} \cdot \sqrt{n}$.
- Work at level 2: $2\sqrt{n}$.
- Work at level k: $\sqrt{2^k} \cdot \sqrt{n}$.
- Total work: $\sum_{k=0}^{\log_2 n} \sqrt{2^k} \cdot \sqrt{n} = O(n)$

Three cases

Decreasing Work

$$T(n) = 3T(n/4) + O(n)$$

The work done at the root is $O(n)$, while the leaves contribute $O(n^{\log_4 3})$. Since $O(n) > O(n^{\log_4 3})$, the root dominates, leading to $O(n)$ complexity.

Equal Work

$$T(n) = 4T(n/4) + O(n)$$

Here, the work done at the root is $O(n)$ and the leaves contribute $O(n^{\log_4 4}) = O(n)$. Since both terms are equal, an extra logarithmic factor appears, giving $O(n \log n)$ complexity.

Increasing Work

$$T(n) = 5T(n/4) + O(n)$$

The root contributes $O(n)$, while the leaves contribute $O(n^{\log_4 5})$. Since $O(n) < O(n^{\log_4 5})$, the leaves dominate, leading to $O(n^{\log_4 5})$ complexity.

A general result

Theorem (Master theorem)

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + f(n),$$

- *If $f(n) = O\left(n^{\log_b a - \epsilon}\right)$ for some constant $\epsilon > 0$, then $T(n) = \Theta\left(n^{\log_b a}\right)$.*
- *If $f(n) = \Theta\left(n^{\log_b a}\right)$, then $T(n) = \Theta\left(n^{\log_b a} \lg n\right)$.*
- *If $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.*

Master theorem

The three cases correspond to

1. the weight increases geometrically from node to leaves.
2. the weight is roughly equal in each layer
3. the weight decreases geometrically from node to leaves.

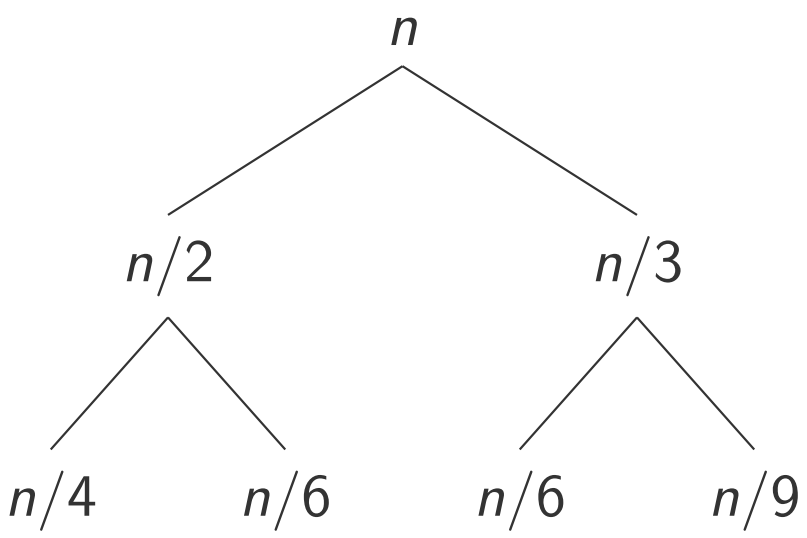
Find the Complexity

Consider the following recurrence relations and determine their complexities:

- $T(n) = 4T(n/2) + O(n^2)$
- $T(n) = 7T(n/2) + O(n^2)$
- $T(n) = 5T(n/3) + O(n^3)$
- $T(n) = 5T(n/3) + O(n^{1.5})$
- $T(n) = 5T(n/2) + O(n^2)$
- $T(n) = 9T(n/3) + O(n^2)$
- $T(n) = 2T(n/2) + O(n \log n)$
- $T(n) = 3T(n/3) + O(n^2 - n \log n)$
- $T(n) = 2T(n/2 + 1) + O(n)$

Example: Unbalanced Tree - Decreasing Work

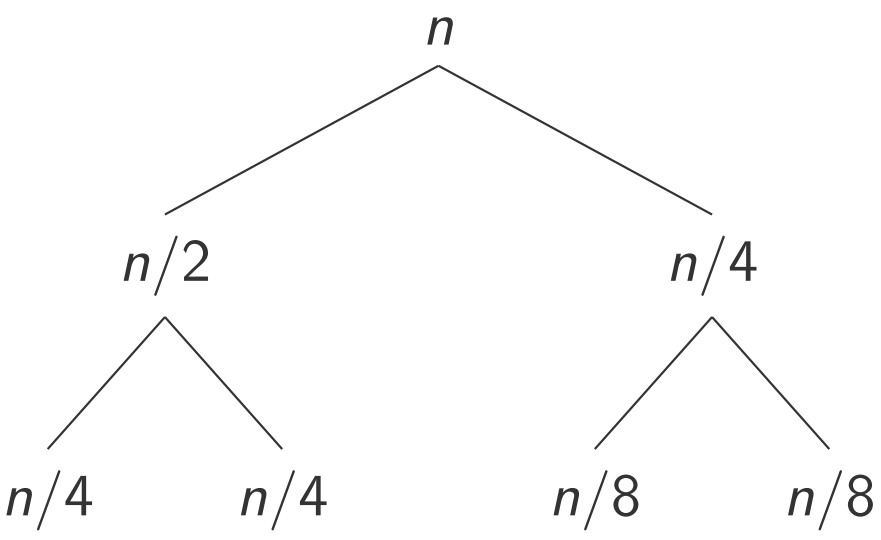
- Consider $T(n) = T(n/2) + T(n/3) + \Theta(n)$.
- The work done at each level is decreasing geometrically.



- The left branch height is $\log_2 n$ which is bigger then right one with height $\log_3 n$
- The total work sums to $\Theta(n)$.

Example: Unbalanced Tree - Equal Work

- Consider $T(n) = 2T(n/4) + T(n/2) + O(n)$.
- The work done at each level is balanced across subproblems.



- The left branch height is $\log_2 n$ which is bigger then right one with height $\log_3 n$
- The total work sums is at least $(n \log_3 n)$ and at most $(n \log_2 n)$, hence it is $\Theta(n \log n)$.

Change of Variables

Solve the recurrence:

$$T(n) = 2T(\sqrt{n}) + \log n$$

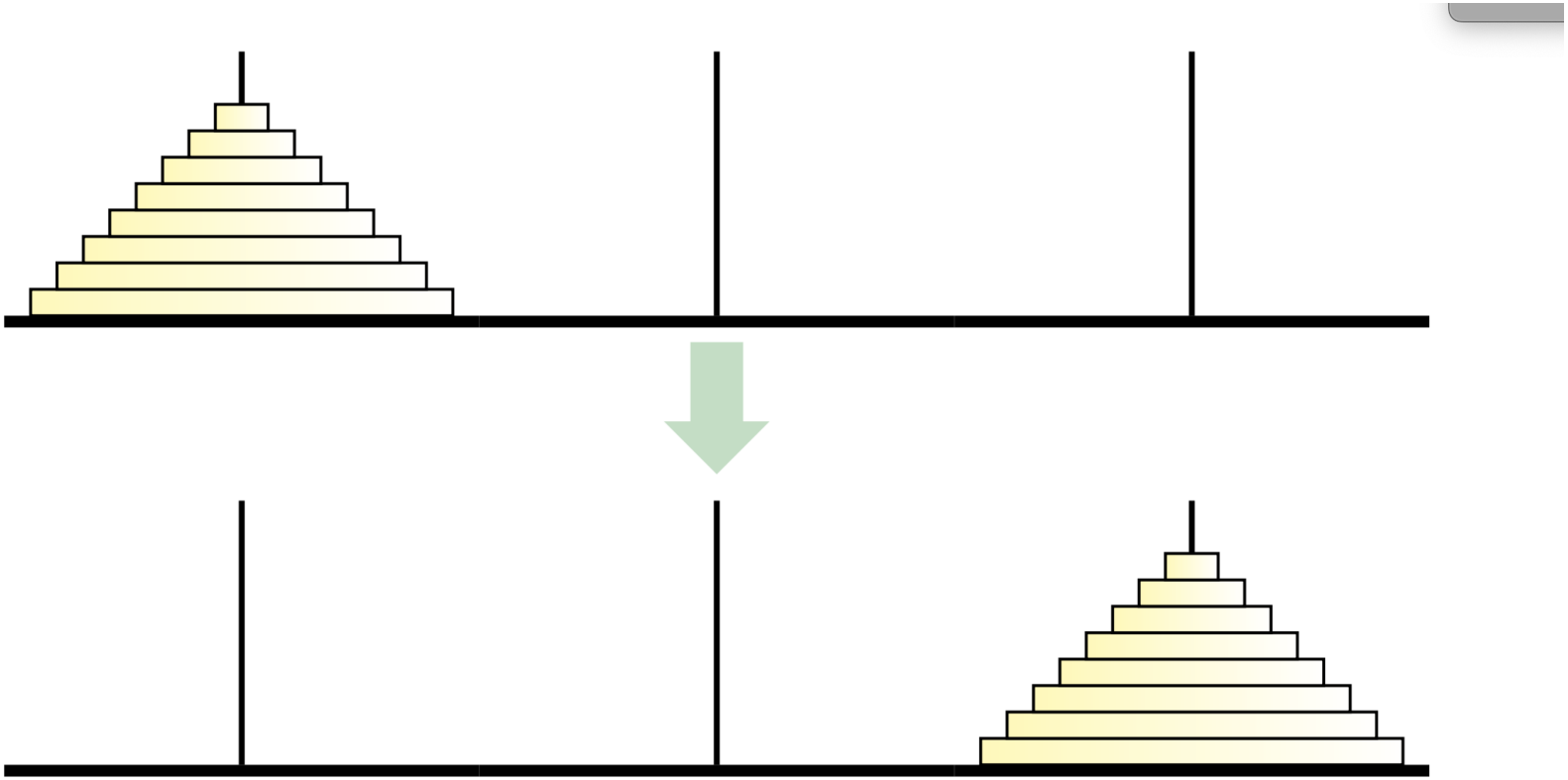
- Use substitution: Let $n = 2^m$.
- Now, the recurrence becomes:

$$S(m) = 2S(m/2) + m$$

- This is a standard recurrence.
- Solution: $S(m) = O(m \log m) \Rightarrow T(n) = O(\log n \log \log n)$.

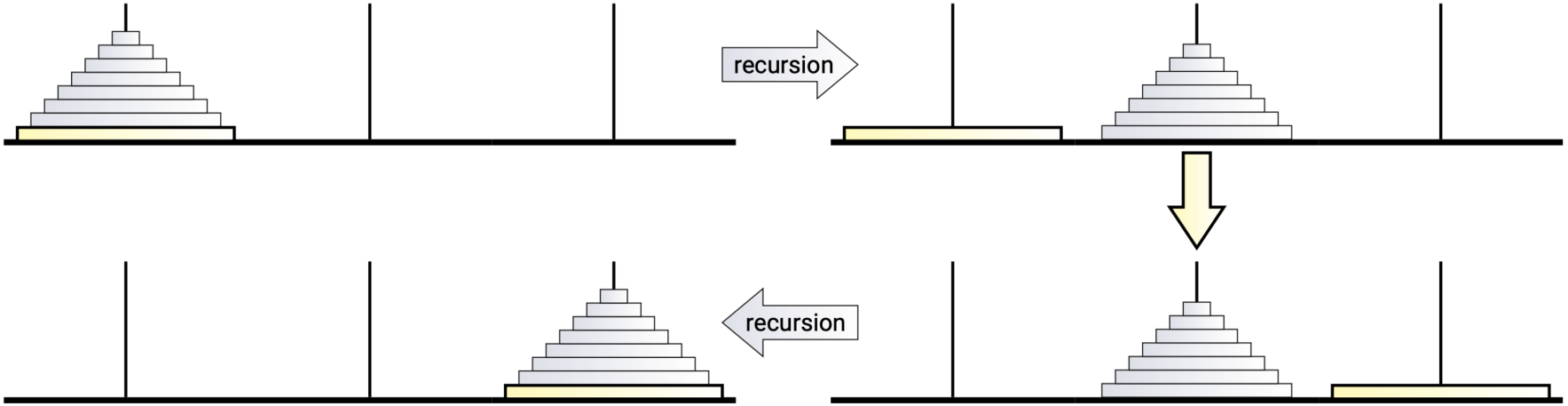
Tower of Hanoi

How can we move a tower of n disks from one peg to another, using a third spare peg as an occasional placeholder, without ever placing a disk on top of a smaller disk?



Tower of Hanoi

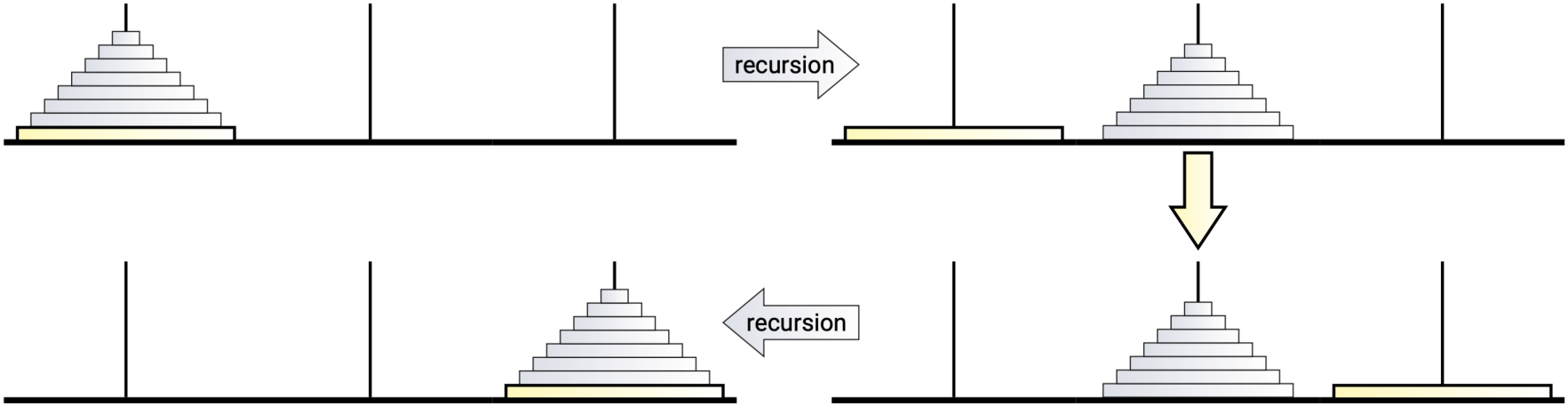
How can we move a tower of n disks from one peg to another, using a third spare peg as an occasional placeholder, without ever placing a disk on top of a smaller disk?



RECURSION!

Tower of Hanoi

How can we move a tower of n disks from one peg to another, using a third spare peg as an occasional placeholder, without ever placing a disk on top of a smaller disk?



RECURSION!

$$T(n) = 2T(n-1) + 1 \implies T(n) = 2^n - 1$$

Searching

Given a list of integers, how long does it take to find if an integer is in the list?

Searching

Given a list of integers, how long does it take to find if an integer is in the list?

If there is no structure to the list, you have to look at all the integers so you need $\Theta(n)$ time.

Searching

Given a sorted list of integers, how long does it take to find if an integer is in the list?

Searching

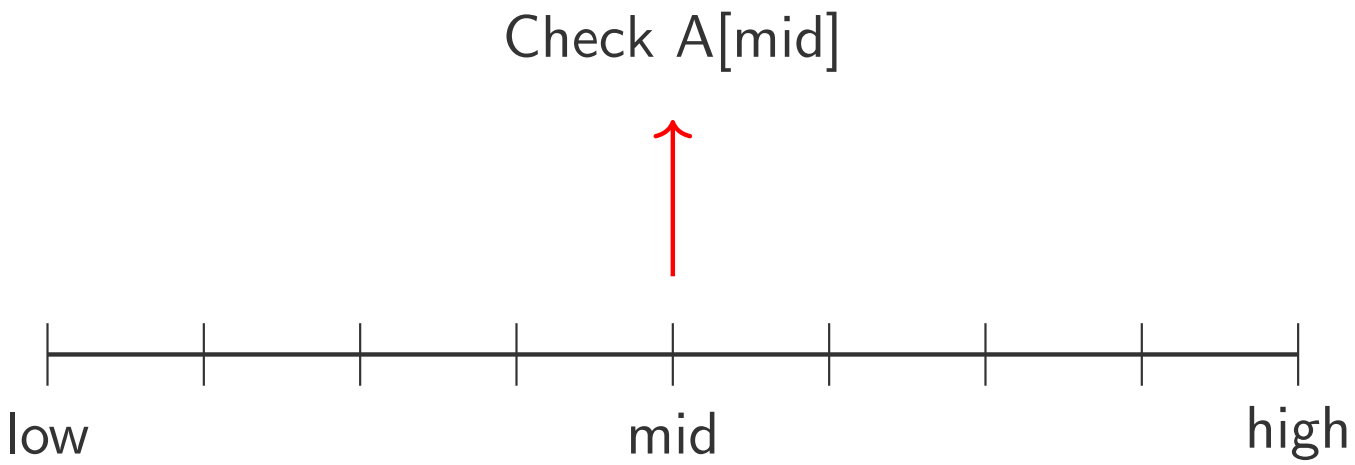
Given a sorted list of integers, how long does it take to find if an integer is in the list?

Divide and Conquer! Check if could be in the left half or the right half.

Binary Search

You don't have to search in both halves!

- Instead of searching in both halves, we compare with the middle element and recurse in only one half.
- This reduces the number of recursive calls, significantly improving efficiency.



Binary Search

1. Initialize two pointers: $left \leftarrow 1$ and $right \leftarrow n$
2. While $left \leq right$:
   2.1 Calculate the midpoint: $mid \leftarrow \left\lfloor \frac{left+right}{2} \right\rfloor$
   2.2 If $A[mid] = x$, return `Found`
   2.3 If $x < A[mid]$, set $right \leftarrow mid - 1$ (search the left half)
   2.4 If $x > A[mid]$, set $left \leftarrow mid + 1$ (search the right half)
3. If the loop ends without finding x, return `Not Found`

- The recurrence relation for Binary Search is:

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

- This results in a time complexity of $O(\log n)$.

Searching

Given a list of integers, how long does it take to find if an integer is in the list?

Two-Sum Problem

Given an array of integers and an integer target, return indices of the two numbers such that they add up to the target.

Three Sum

Given an array of n distinct integers, find three that sum to 0.

Two-Sum Problem

Given an array of integers and an integer target, return indices of the two numbers such that they add up to the target.

Two-Sum Problem

Given an array of integers and an integer target, return indices of the two numbers such that they add up to the target.

Simple Algorithm: The idea is to check every pair (i, j) until we find one that works; if none are found, return "no solution"

1. For each index $i, 0 \leq i < n$
2. for each index $j \neq i$
3. if $A[i] + A[j] = T$ return (i, j)
4. Return "no solution"

Analysis: Overall the algorithm checks $\Theta\left(n^2\right)$ different pairs (i, j)

Two-Sum Problem

Given an array of integers and a target integer, return the indices of two numbers such that they add up to the target.

Better Algorithm:

1. Sort the array with Merge Sort
2. left $\leftarrow 0$
3. right $\leftarrow n - 1$
4. While (left $<$ right)
   4.1 if $(\mathrm{A}[\text{ left }] + \mathrm{A}[\text{ right }] = \mathrm{T})$, return pair (left, right)
   4.2 if $(\mathrm{A}[\text{ left }] + \mathrm{A}[\text{ right }] < \mathrm{T})$, do left $\leftarrow$ left $+1$
   4.3 if $(\mathrm{A}[\text{ left }] + \mathrm{A}[\text{ right }] > \mathrm{T})$, do right $\leftarrow$ right -1
5. return "no solution found"

Analysis: The algorithm iterates n times through the `while` loop, as each iteration decreases right $-$ left by one. Therefore, the time complexity is $\Theta(n)$ due to the linear scan after sorting, rather than $\Theta(n^2)$. With sorting it is $\Theta(n \log n)$. If we have a sorted array is $\Theta(n)$.

Three Sum

Given an array of n distinct integers, find three that sum to 0.

Three Sum

Given an array of n distinct integers, find three that sum to 0.

Another Example: Finding Max Profit

You are given an array prices where prices[i] is the price of a given stock on the ith day. You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock. Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Input: Array $P[0 \ldots n-1]$ of length n
Output: Max $\{P[j] - P[i]\}$ over all pairs i, j with $j > i$. If $P[j] - P[i]$ is always negative then output 0 (don't buy).

Another Example: Finding Max Profit

Input: Array $P[0 \ldots n-1]$ of length n

Output: Max $\{P[j] - P[i]\}$ over all pairs i, j with $j > i$. If $P[j] - P[i]$ is always negative then output 0 (don't buy).

Trivial Solution: Try all pairs (i, j) with $i > j$. So $\Theta(n^2)$.

Finding Max Profit

Input: Array $P[0 \ldots n-1]$ of length n

Output: Max $\{P[j] - P[i]\}$ over all pairs i, j with $j > i$. If $P[j] - P[i]$ is always negative then output 0 (don't buy).

Better Algorithm

- If $\text{len}(P) = 1$, return 0.
- $P_1 \leftarrow P[0], \ldots, P[\frac{n}{2} - 1]$
- $P_2 \leftarrow P[\frac{n}{2}], \ldots, P[n-1]$
- $x_1 \leftarrow \text{MaxProfit}(P_1)$
- $x_2 \leftarrow \text{MaxProfit}(P_2)$
- $x_3 \leftarrow \text{FindMax}(P_2) - \text{FindMin}(P_1)$
- Return $\max\{x_1, x_2, x_3\}$

$$T(n) = 2T(n/2) + O(n)$$

So the algorithm runs in $O(n \log n)$.

Finding Max Profit

Idea: Recursive algorithms can be better if they return more information.

Input: Array $P[0 \ldots n-1]$ of length n
Output: Max $\{P[j] - P[i]\}$ over all pairs i, j with $j > i$. If $P[j] - P[i]$ is always negative then output 0 (don't buy). Minimum and Maximum of P.

Finding Max Profit

Input: Array $P[0 \ldots n-1]$ of length n

Output: Max $\{P[j] - P[i]\}$ over all pairs i, j with $j > i$. If $P[j] - P[i]$ is always negative then output 0 (don't buy). Minimum and Maximum of P.

- If $(\text{len}(P) = 1)$, return $0, P[0], P[0]$.
- $P_1 \leftarrow P[0], \ldots, P[\frac{n}{2} - 1]$
- $P_2 \leftarrow P[\frac{n}{2}], \ldots, P[n-1]$
- $\text{PRFT}_1, MN_1, MX_1 \leftarrow \text{MaxProfitX}(P_1)$
- $\text{PRFT}_2, MN_2, MX_2 \leftarrow \text{MaxProfitX}(P_2)$
- $\text{PRFT}_3 \leftarrow MX_2 - MN_1$
- $\text{PRFT} \leftarrow \max\{\text{PRFT}_1, \text{PRFT}_2, \text{PRFT}_3\}$
- $MX \leftarrow \max\{MX_1, MX_2\}$
- $MN \leftarrow \min\{MN_1, MN_2\}$
- Return PRFT, MN, MX.

$$T(n) \leq 2T(n/2) + 1$$

So we have $T(n) = O(n)$.

Selection Problem

Problem: Select the k-th smallest element in an array of size n.

Applications: Find the median, top quantiles, etc.

Key Definitions:

- Element of rank k is the k-th smallest in A.
- Median = element of rank $\lceil n/2 \rceil$.

Naive Solution: Sort array and select, $\Theta(n \log n)$ time.

Algorithm for SELECT(A, k)

Input: Array A of length n and integer k.

Steps:

- Find median m of A.
- Partition A around m into A_1 (elements $< m$) and A_2 (elements $> m$).
- Recur based on k:
  - If $k = \lceil n/2 \rceil$, return m.
  - If $k < \lceil n/2 \rceil$, recur on A_1.
  - If $k > \lceil n/2 \rceil$, recur on A_2 with $k' = k - \lceil n/2 \rceil$.

Partitioning in $\Theta(n)$

Partition(A, m):

- Input: Array A and pivot m.
- Output: Two arrays, A_1 (elements $< m$) and A_2 (elements $> m$).
- Time complexity: $\Theta(n)$.

Example: SELECT(A, 3)

Array: $A = [7, 11, 2, 9, 4, 6, 1, 15, 5]$

- Median of A: $m = 6$.
- Partition: $A_1 = [2, 4, 1, 5]$, $A_2 = [7, 11, 9, 15]$.
- Recur on A_1 for $k = 3$. Output is 4.

Running Time Analysis

In Fantasy Land:

- FindMedian in $\Theta(n)$.
- $T(n) = T(n/2) + O(n)$.
- Solves to $T(n) = \Theta(n)$.

In Reality:

- Use a pivot selection strategy that gives a good split.
- Approximate median leads to $T(n) = O(n)$.

General Framework for SELECT

Framework:

- $p \leftarrow$ ChoosePivot(A)
- $A_1, A_2 \leftarrow$ Partition(A, p)
- Recur based on k:
  - If $k < \text{len}(A_1) + 1$, return Select$(A1, k)$
  - If $k = \text{len}(A_1) + 1$, return p
  - If $k > \text{len}(A_1) + 1$, return Select$(A2, k - \text{len}(A_1) - 1)$

Deterministic SELECT in $\Theta(n)$

Steps:

- Split A into groups of 5.
- Find the median of each group.
- Recursively find the median of medians.
- Partition around the median of medians.
- Recur on the appropriate side.

Running Time:

- $T(n) \leq T(n/5) + T(7n/10) + O(n)$, solves to $\Theta(n)$.